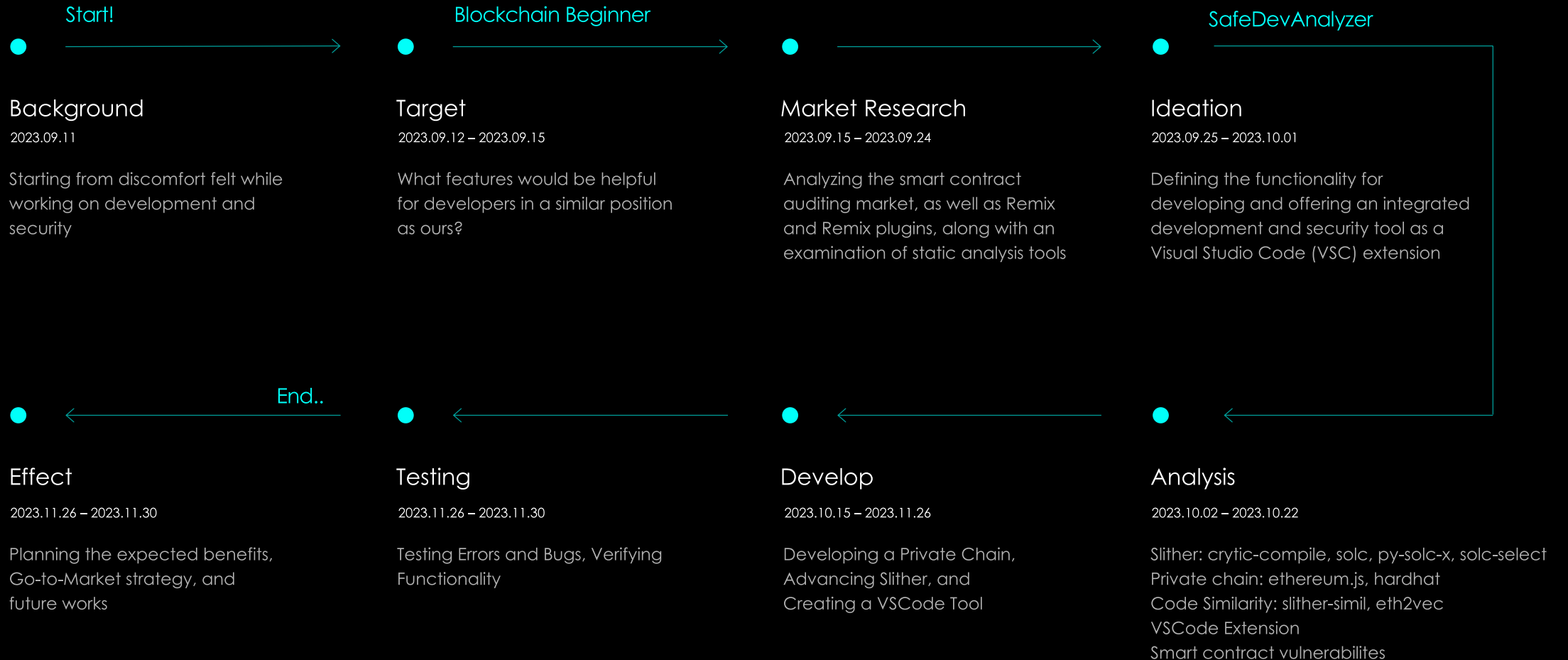


ANTIBUG

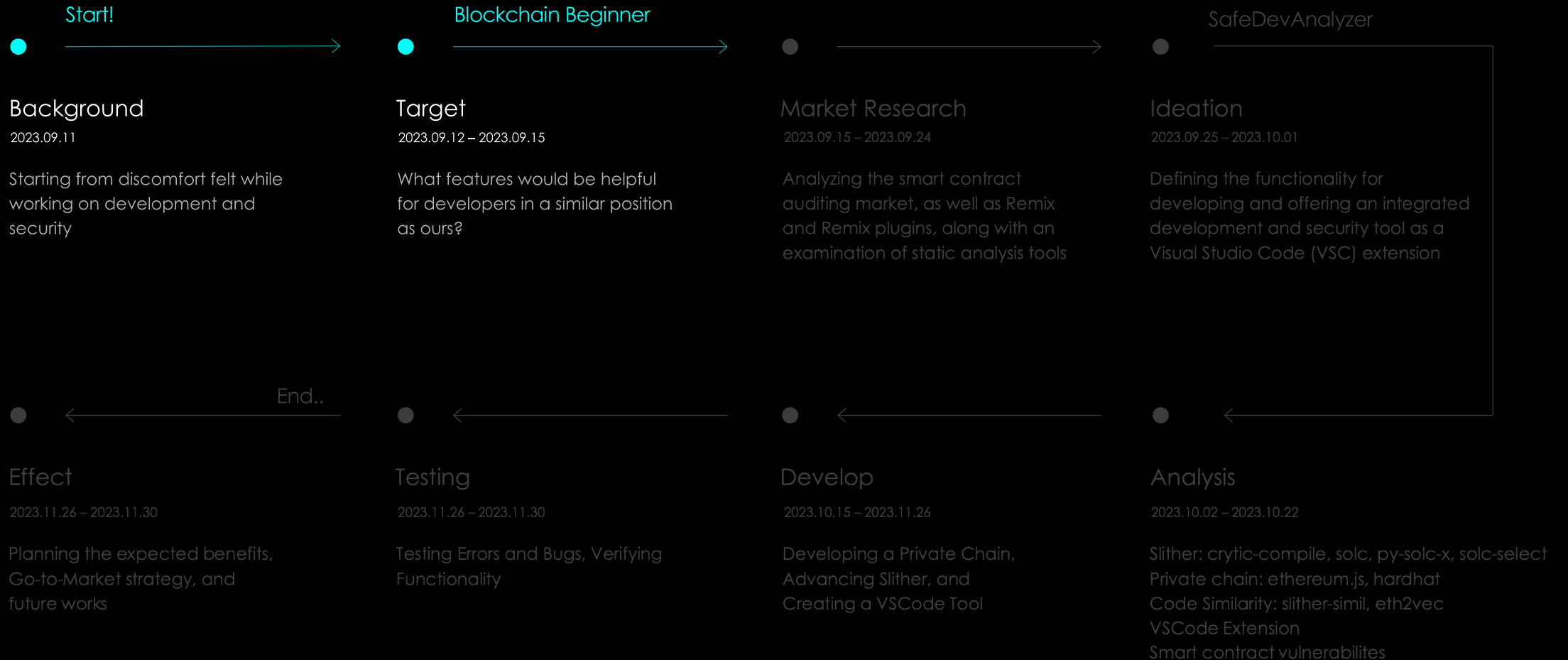
S a f e D e v A n a l y z e r

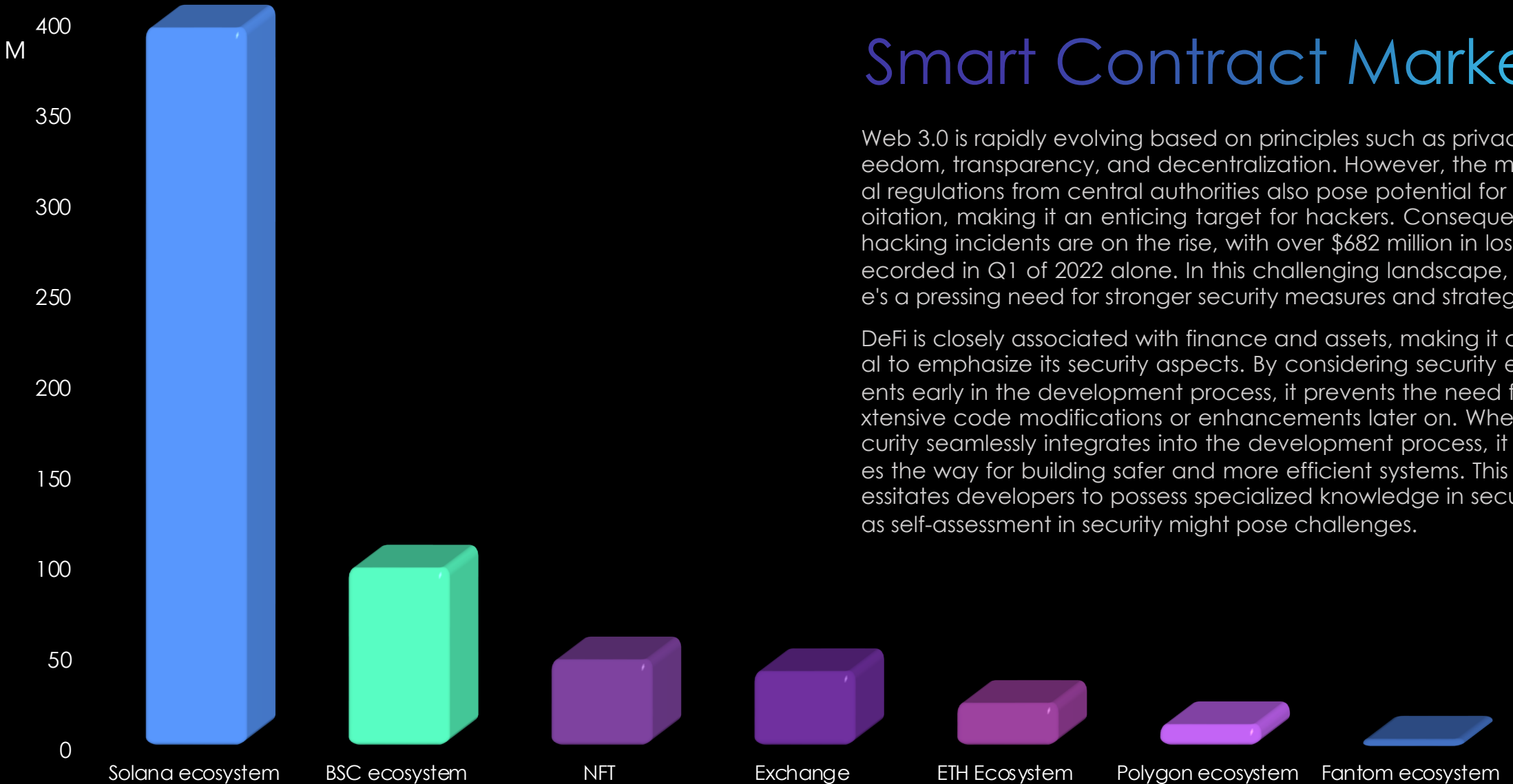
PROTOCOL CAMP
by  DREAMPLUS X *#HASHED*

Antibug Progress



Antibug Progress





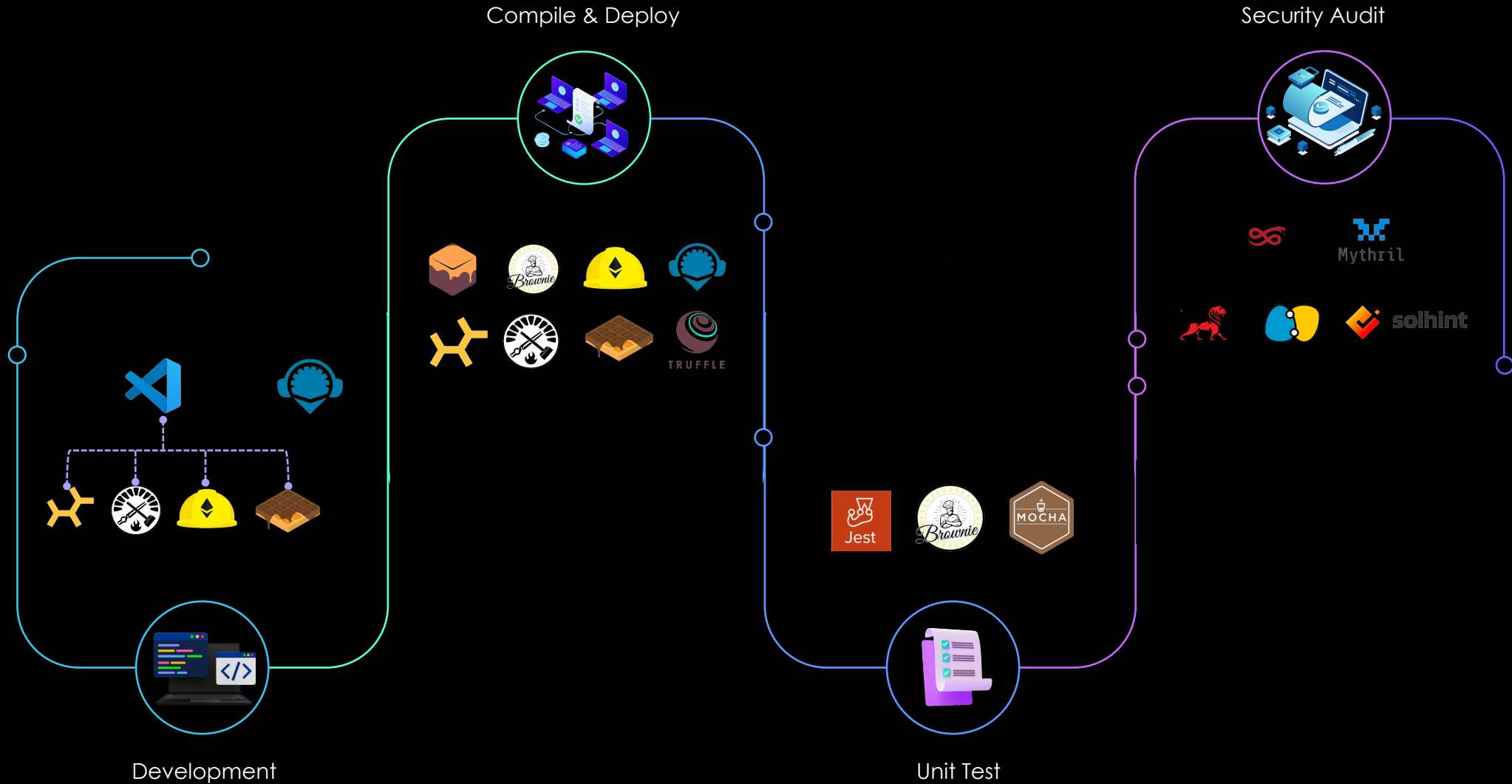
Smart Contract Market

Web 3.0 is rapidly evolving based on principles such as privacy, freedom, transparency, and decentralization. However, the minimal regulations from central authorities also pose potential for exploitation, making it an enticing target for hackers. Consequently, hacking incidents are on the rise, with over \$682 million in losses recorded in Q1 of 2022 alone. In this challenging landscape, there's a pressing need for stronger security measures and strategies.

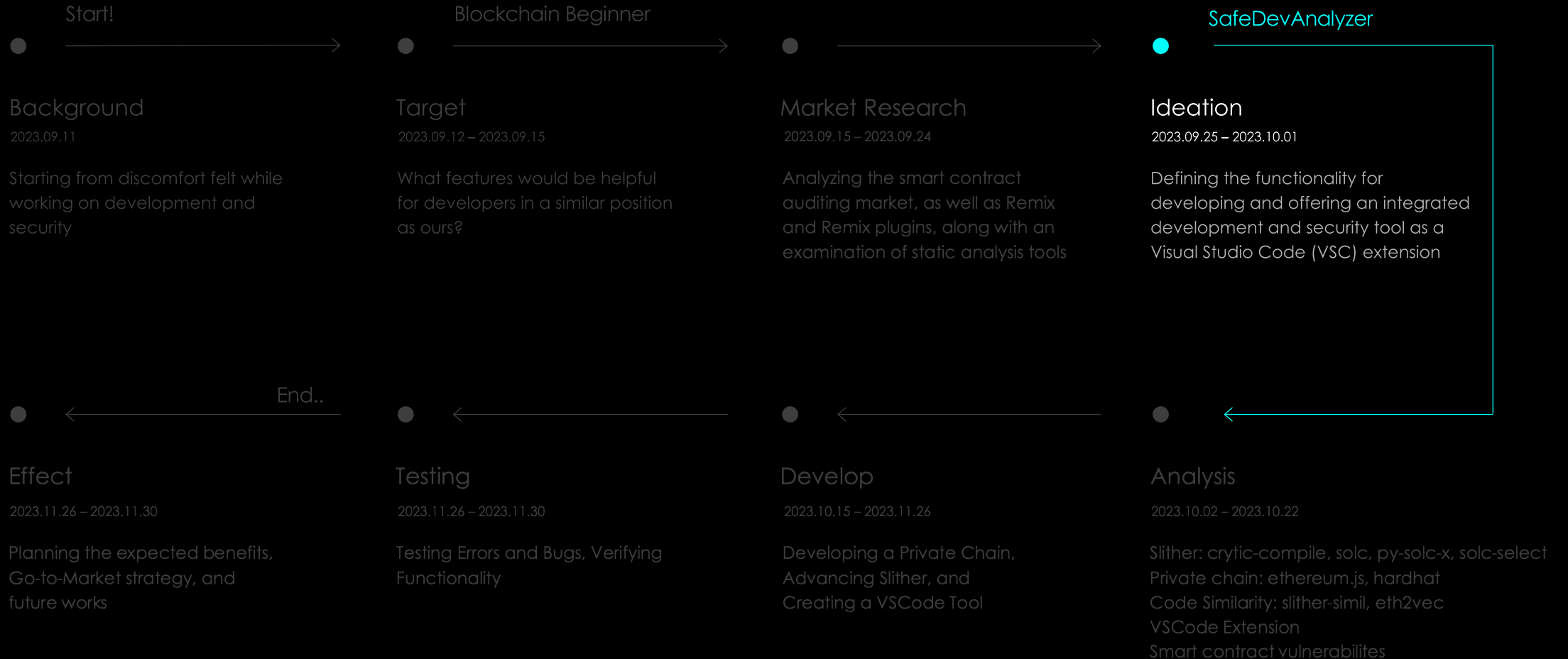
DeFi is closely associated with finance and assets, making it crucial to emphasize its security aspects. By considering security elements early in the development process, it prevents the need for extensive code modifications or enhancements later on. When security seamlessly integrates into the development process, it paves the way for building safer and more efficient systems. This necessitates developers to possess specialized knowledge in security, as self-assessment in security might pose challenges.

Market Research:

Smart Contract Development Process



Antibug Progress



Ideation

We aim to seamlessly integrate static analysis tools into the development process to enhance security aspects. Furthermore, we intend to raise security awareness among developers and emphasize



Development



VSCode



Security

SafeDevAnalyzer



with Private chain

Local Testing

- Compile and deploy without wallet connection.
- Interact with smart contracts using 10 generated accounts and 100 ETH



with SafeDevAnalyzer

Security Check

- Detecting vulnerabilities within contracts using static analysis tool
- Providing contract analysis information such as call-graph and function Info

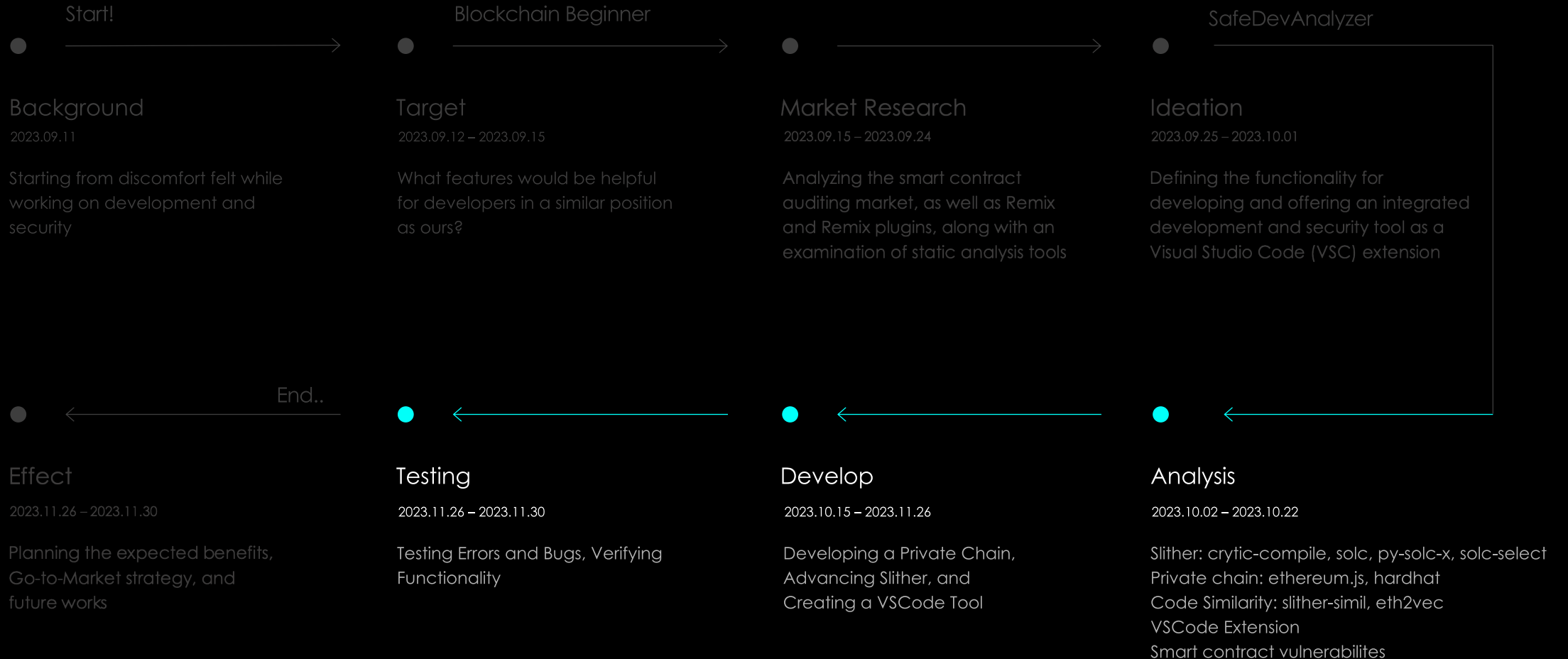


with SafeDevAnalyzer

Audit Report

- Detailed reporting of vulnerability detection results
- Exporting in markdown format

Antibug Progress



AntiChain

✓ Shanghai Hardfork(2023-04-12)

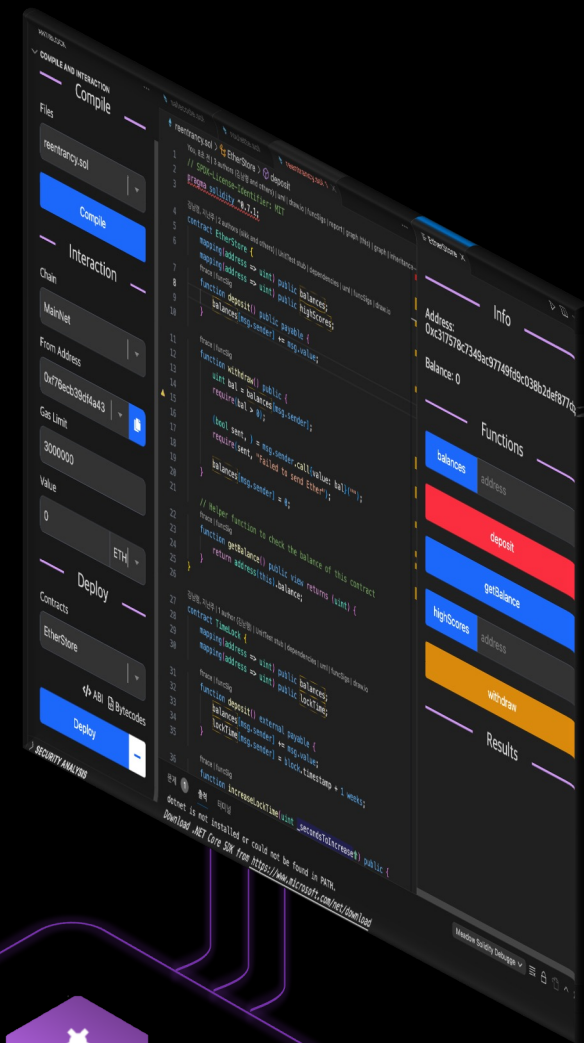
✓ EIP-1559, EIP-4895

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.7.1;

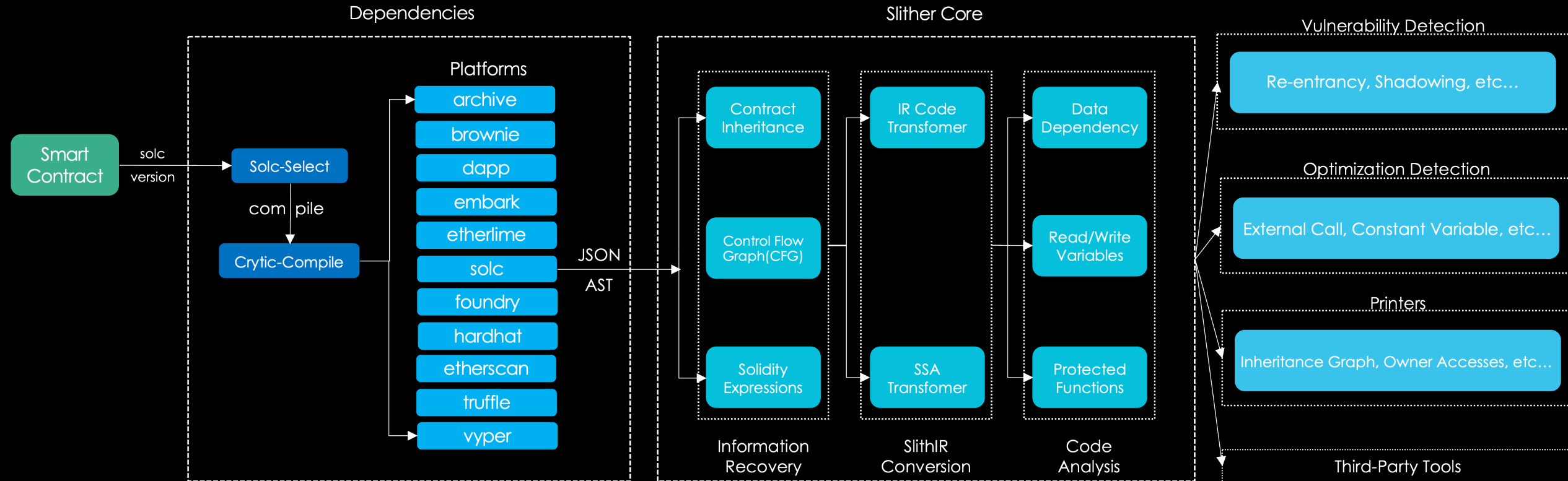
contract EtherStore {
    mapping(address => uint) public balances;
    mapping(address => uint) public highScores;
    function deposit() public payable {
        balances[msg.sender] += msg.value;
    }

    function withdraw() public {
        uint bal = balances[msg.sender];
        require(bal > 0);
        (bool sent, ) = msg.sender.call{value: bal}("");
        require(sent, "Failed to send Ether");
        balances[msg.sender] = 0;
    }

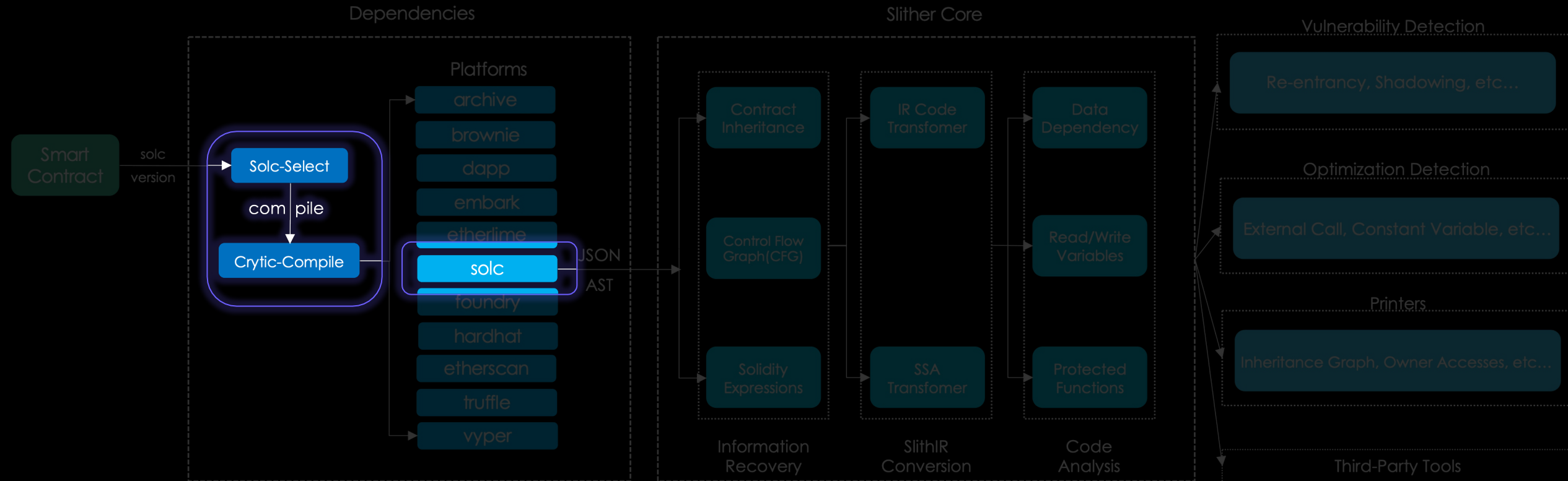
    // Helper function to check the balance of this contract
    function getBalance() public view returns (uint) {
        return address(this).balance;
    }
}
```



Slither Core Process



Slither Core Process



Uncomfortable usage



1 solc-select install [solc version]

Install the solc binary corresponding to the pragma version in the smart contract

2 solc-select use [solc version]

Set the installed solc version as the global solc version

3 slither [detect option] [file]

Run Slither



Uncomfortable usage

| Slither

```
sikk@gimnamlyeong-ui-MacBookPro test % slither test.sol
Compilation warnings/errors on test.sol:
Error: Source file requires different compiler version (current compiler is
0.8.21+commit.d9974bed.Darwin.appleclang) - note that nightly builds are considered to be strictly
less than the released version
--> test.sol:2:1:
  |
2 | pragma solidity ^0.7.1;
  | ~~~~~
sikk@gimnamlyeong-ui-MacBookPro test % solc-select install 0.7.2
Installing solc '0.7.2'...
Version '0.7.2' installed.

sikk@gimnamlyeong-ui-MacBookPro test % solc-select use 0.7.2
Switched global version to 0.7.2

sikk@gimnamlyeong-ui-MacBookPro test % slither test.sol
INFO:Detectors:
Reentrancy in EtherStore.withdraw() (test.sol#11-19):
  External calls:
  - (sent) = msg.sender.call{value: bal}() (test.sol#15)
  State variables written after the call(s):
  - balances[msg.sender] = 0 (test.sol#18)
  EtherStore.balances (test.sol#5) can be used in cross function reentrancies:
  - EtherStore.balances (test.sol#5)
  - EtherStore.deposit() (test.sol#7-9)
  - EtherStore.withdraw() (test.sol#11-19)
```

| SafeDevAnalyzer

```
sikk@gimnamlyeong-ui-MacBookPro test % antibug detect test.sol
INFO:Detectors:
Reentrancy in EtherStore.withdraw() (test.sol#11-19):
  External calls:
  - (sent) = msg.sender.call{value: bal}() (test.sol#15)
  State variables written after the call(s):
  - balances[msg.sender] = 0 (test.sol#18)
  EtherStore.balances (test.sol#5) can be used in cross function reentrancies:
  - EtherStore.balances (test.sol#5)
  - EtherStore.deposit() (test.sol#7-9)
  - EtherStore.withdraw() (test.sol#11-19)
```

Enhancing Slither

Core Logic Optimization

We made efforts to optimize the core logic of Slither. Upon analyzing the code, we identified redundant or unnecessary logic contributed by multiple individuals, which we systematically cleaned up and removed. SafeDevAnalyzer primarily focuses on compiling solidity code. Consequently, we streamlined the logic in alignment with [solc](#) within the suite of platforms offered by [crytic compile](#). As a result, the dependency of slither on solc-select and crytic-compile has been entirely eliminated.



Removing
Duplicate Logic



Removing
Unnecessary Logic



Selecting Necessary Logic
in SafeDevAnalyzer

Enhancing Slither

Core Logic Optimization – crytic compile's core logic

```
def _compile(self, **kwargs: str) -> None:
    custom_build: Union[None, str] = kwargs.get("compile_custom_build", None)
    if custom_build:
        self._run_custom_build(custom_build)

    else:
        if not kwargs.get("skip_clean", False) and not kwargs.get("ignore_compile", False):
            self._platform.clean(**kwargs)
            self._platform.compile(self, **kwargs)

    remove_metadata = kwargs.get("compile_remove_metadata", False)
    if remove_metadata:
        for compilation_unit in self._compilation_units.values():
            for source_unit in compilation_unit.source_units.values():
                source_unit.remove_metadata()
```

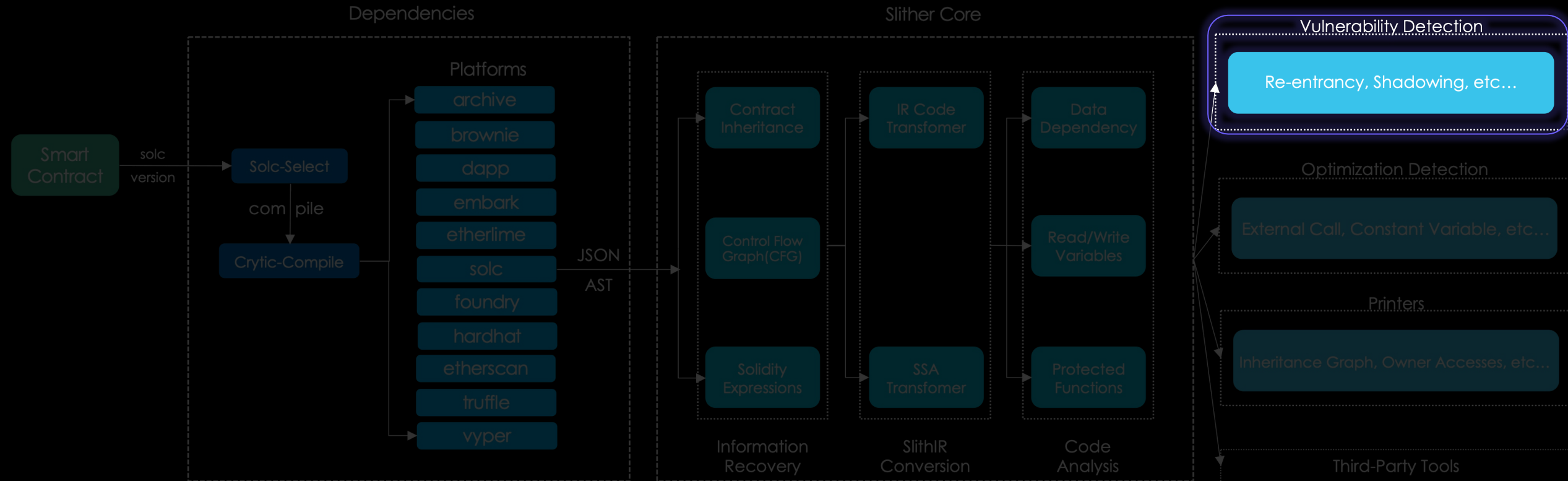
```
def _run_solc(
    compilation_unit: "CompilationUnit", filename: str, solc: str,
    solc_disable_warnings: bool, solc_arguments: Optional[str], solc_remaps:
    Optional[Union[str, List[str]]] = None, env: Optional[Dict] = None, working_dir:
    Optional[Union[Path, str]] = None, force_legacy_json: bool = False, ) -> Dict:
    ...

    compiler_version = compilation_unit.compiler_version
    assert compiler_version
    options = _build_options(compiler_version, force_legacy_json)

    cmd = [solc]
    if solc_remaps:
        if isinstance(solc_remaps, str):
            solc_remaps = solc_remaps.split(" ")
        cmd += solc_remaps
    cmd += [filename, "--combined-json", options]
    try:
        LOGGER.info(
            "'%s' running",
            " ".join(cmd),
        )

        if env:
            process = subprocess.Popen(
                cmd,
                stdout=subprocess.PIPE,
                stderr=subprocess.PIPE,
                executable=shutil.which(cmd[0]),
                env=env,
                **additional_kwargs,
            )
        else:
            process = subprocess.Popen(
                cmd,
                stdout=subprocess.PIPE,
                stderr=subprocess.PIPE,
                executable=shutil.which(cmd[0]),
                **additional_kwargs,
            )
    except OSError as error:
        raise InvalidCompilation(error)
    stdout_, stderr_ = process.communicate()
    stdout, stderr = (
        stdout_.decode(encoding="utf-8", errors="ignore"),
        stderr_.decode(encoding="utf-8", errors="ignore"),
    )
    ...
```

Slither Core Process



Enhancing **Slither**



Additional Information

Remove unnecessary results and provide additional information necessary for vulnerability interpretation.



Audit Reports

Provide vulnerability detection results within the code along with vulnerability description, background, exploit scenario, recommendation, and reference



Detector Rules

Improve the vulnerability detection rules provided by slither

Enhancing Slither



Additional Information

Remove unnecessary results and provide additional information necessary for vulnerability interpretation.



Audit Reports

Provide vulnerability detection results within the code along with vulnerability [description](#), [background](#), [exploit scenario](#), [recommendation](#), and [reference](#)



Detector Rules

Improve the vulnerability detection rules provided by slither

Enhancing Slither



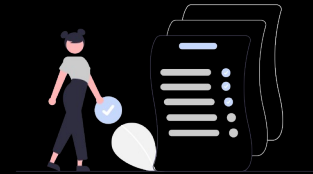
Additional Information

Remove unnecessary results and provide additional information necessary for vulnerability interpretation.



Audit Reports

Provide vulnerability detection results within the code along with vulnerability description, background, exploit scenario, recommendation, and reference



Detector Rules

Improve the vulnerability detection rules provided by slither

Enhancing Slither

Enhancing Detect Rules

Slither

weak-prng

✓ Use Blockhash?

✓ Use modulo calculation?

timestamp

✓ Use block.timestamp?

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
contract GuessTheRandomNumber {
    constructor() payable {}

    uint answer = uint(
        keccak256(abi.encodePacked(blockhash(block.number - 1), block.difficulty))
    );

    if (_guess == answer) {
        (bool sent, ) = msg.sender.call{value: 1 ether}("");
        require(sent, "Failed to send Ether");
    }
}
```

SafeDevAnalyzer

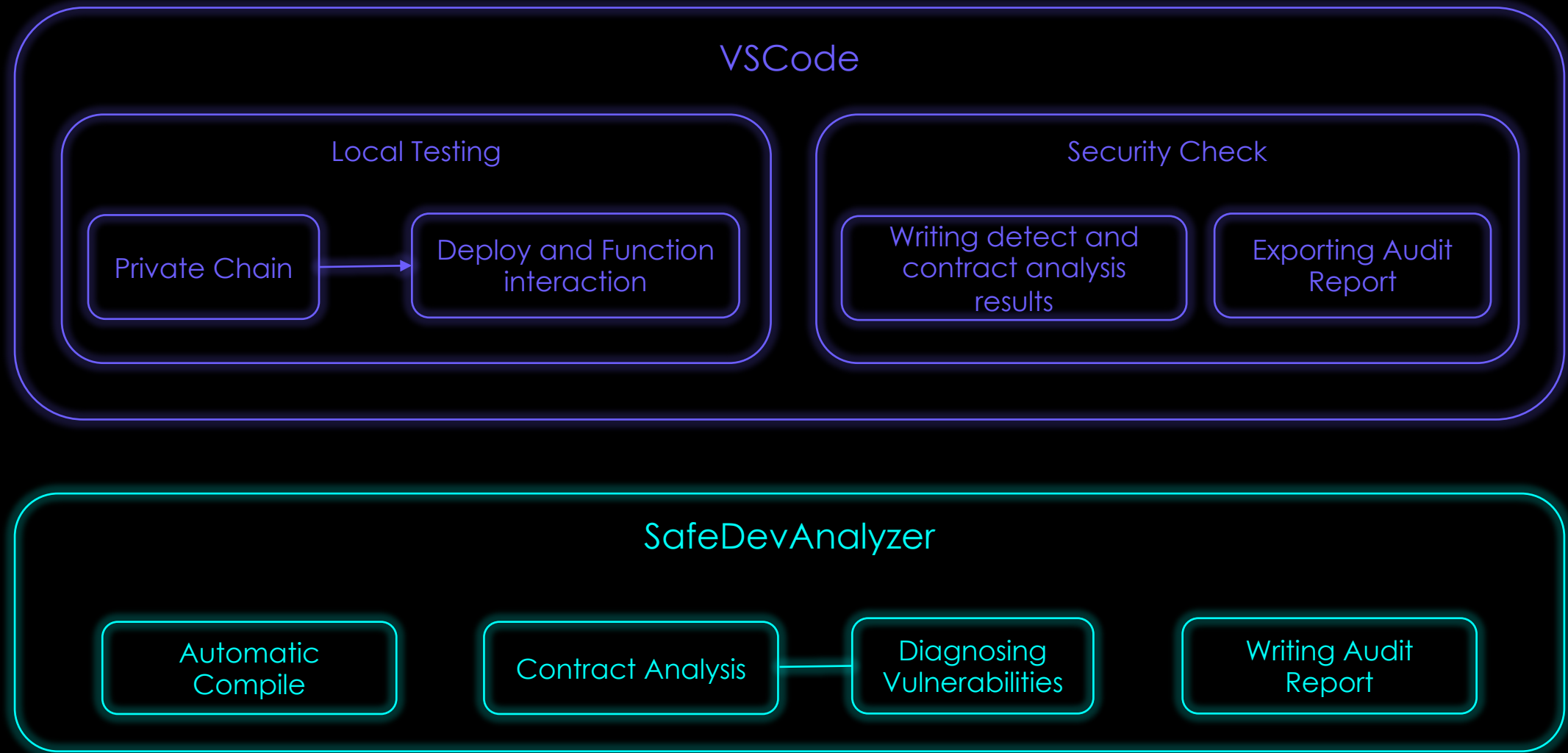
✓ Use blockhash, keccak256?

✓ Use block variables?

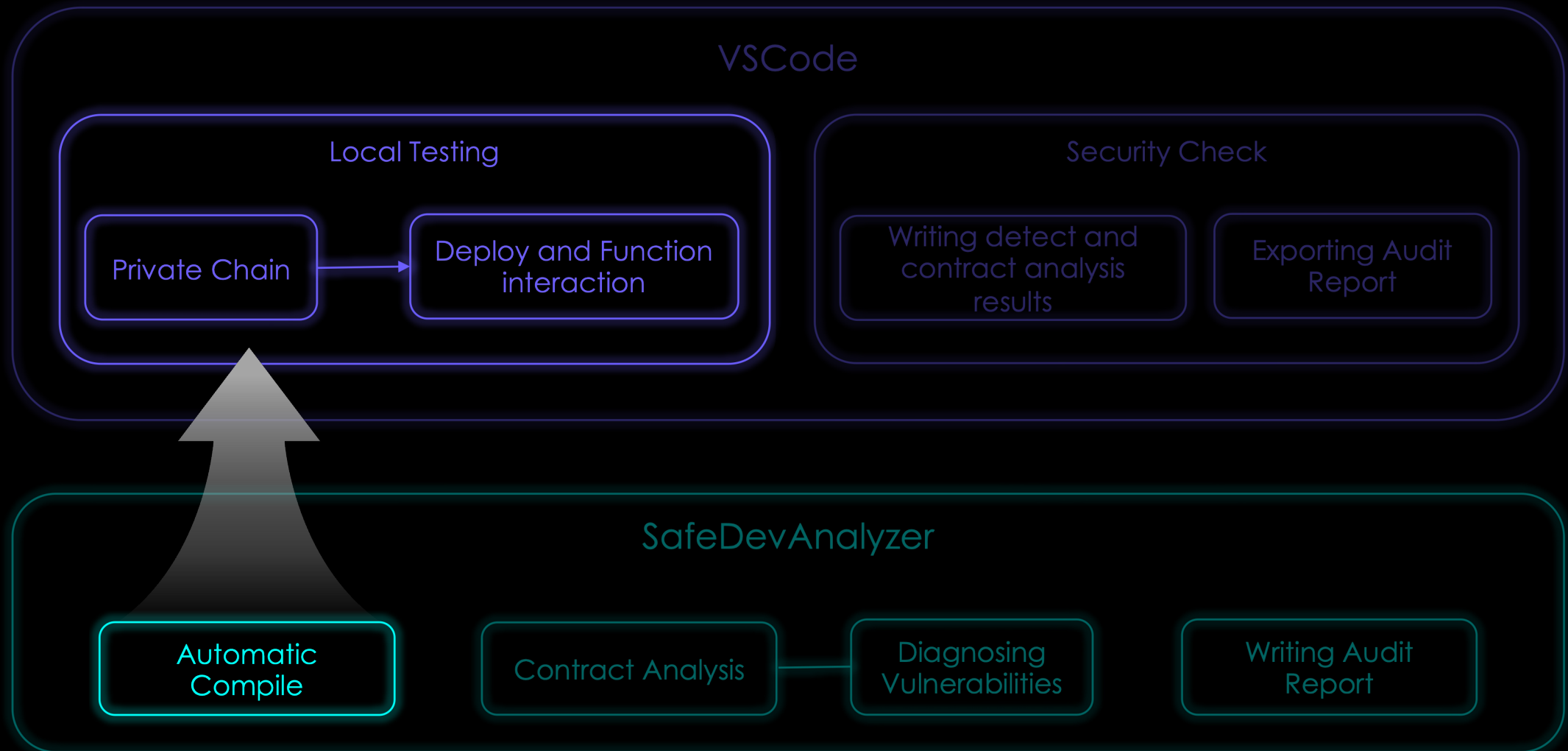
✓ Use modulo calculation?

✓ Are variables that store random numbers used in conditional statements?

SafeDevAnalyzer



SafeDevAnalyzer



Compile & Interaction

ANTIBLOCK

COMPILER AND INTERACTION

Files

etherstore.sol

Compile

Interaction

Chain

MainNet

From Address

0xdacb6aa0ca6147c...

Gas Limit

3000000

Value

0

ETH

Deploy

Contracts

EtherStore

ABI Bytecodes

Deploy

SECURITY ANALYSIS

etherstore.sol 1, U X

EtherStore

um | draw.io | funcSigs | report | graph (this) | graph | inheritance | parse | flatten

1 // SPDX-License-Identifier: MIT

2 pragma solidity ^0.7.1;

3

4 contract EtherStore {

5 mapping(address => uint) public balances;

6 mapping(address => uint) public highScores;

7 address public owner;

8

9 modifier onlyOwner {

10 require(tx.origin == owner, "Not owner");

11 _;

12 }

13

14 function deposit() public payable {

15 balances[msg.sender] += msg.value;

16 }

17

18 function withdraw(uint bal) public {

19

20 (bool sent,) = msg.sender.call{value: bal}("");

21 require(sent, "Failed to send Ether");

22 balances[msg.sender] -= bal;

23 }

24

25 function guess(uint _guess) public {

26 uint answer = uint(

27 keccak256(abi.encodePacked(blockhash(block.number - 1), block.timestamp))

28);

29

30 if (_guess == answer) {

31 reward(msg.sender);

32 }

33 }

34

35 function reward(address to) public onlyOwner(){

36 (bool sent,) = to.call{value: 1 ether}("");

37 require(sent, "Failed to send Ether");

38 }

39

40

41 function getBalance() public view returns (uint) {

42 return address(this).balance;

43 }

44

45 }

EtherStore X

Info

Address: 0x66bb1048d309ad26d9da9d2b9e94a42939636cbe

Balance: 1000000000000000000

Storage

Key	Value
0x5dca36e7e8b36d8ee886bf5c4e356d3c...	0x884563918244f40000
0x7b55a4e5df376282899e5f39acc83b6...	0x884563918244f40000

Functions

balances

0xdacb6aa0ca6147c8a13d78a13c294c70a389b427

deposit

getBalance

guess

uint256 _guess

highScores

address

owner

reward

address to

withdraw

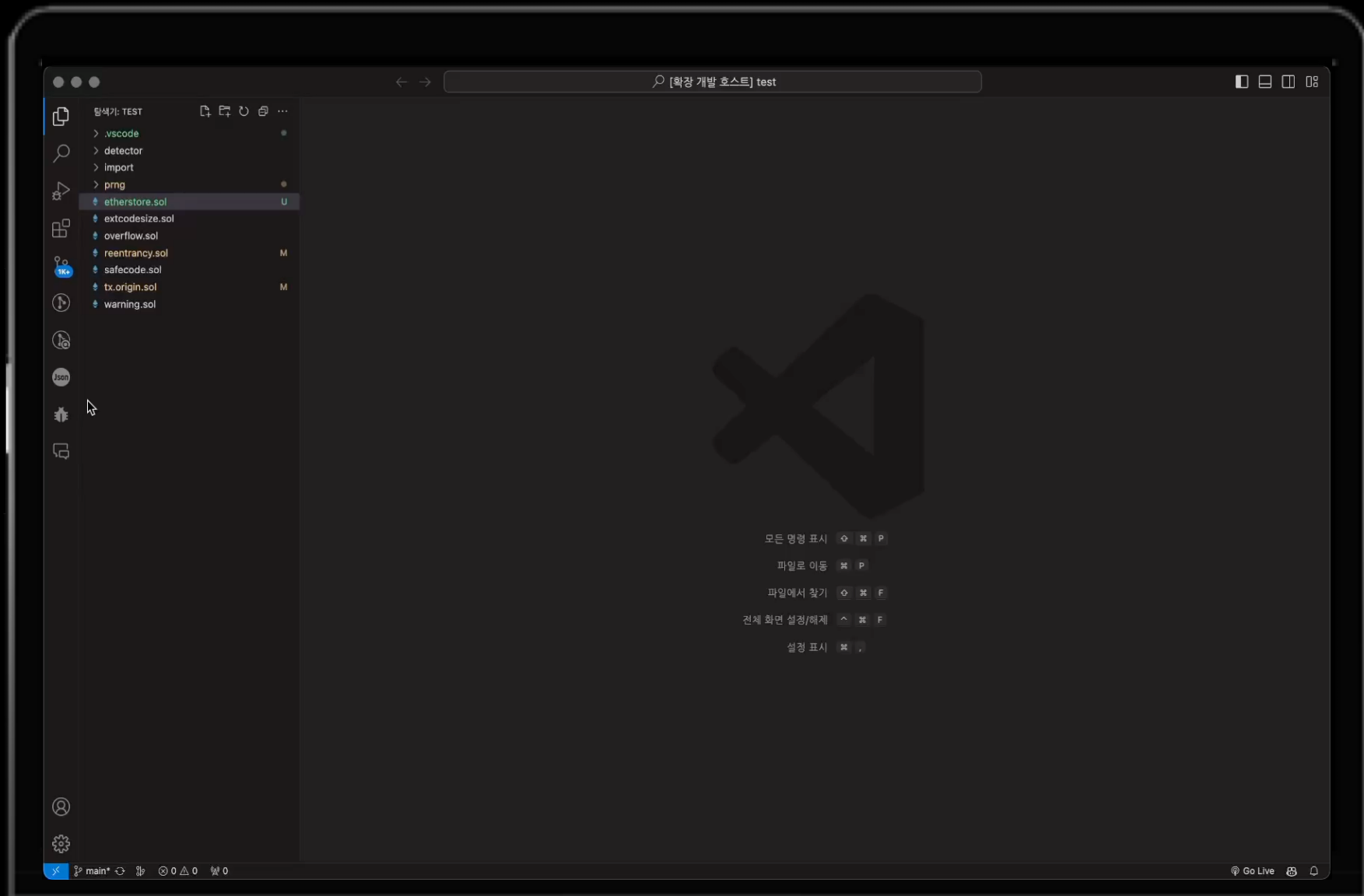
5000000000000000000

Results

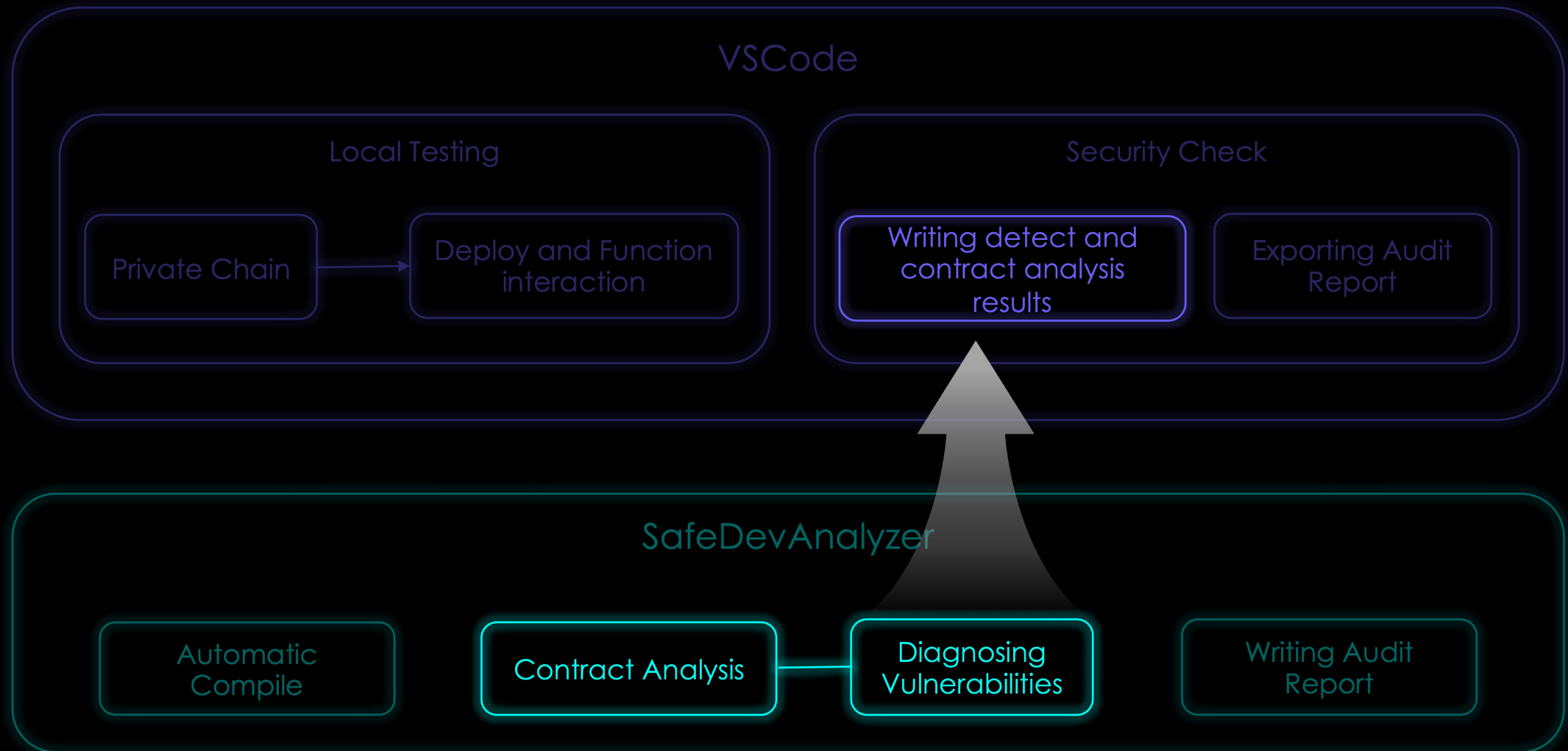
main* 1 0 0

Go Live

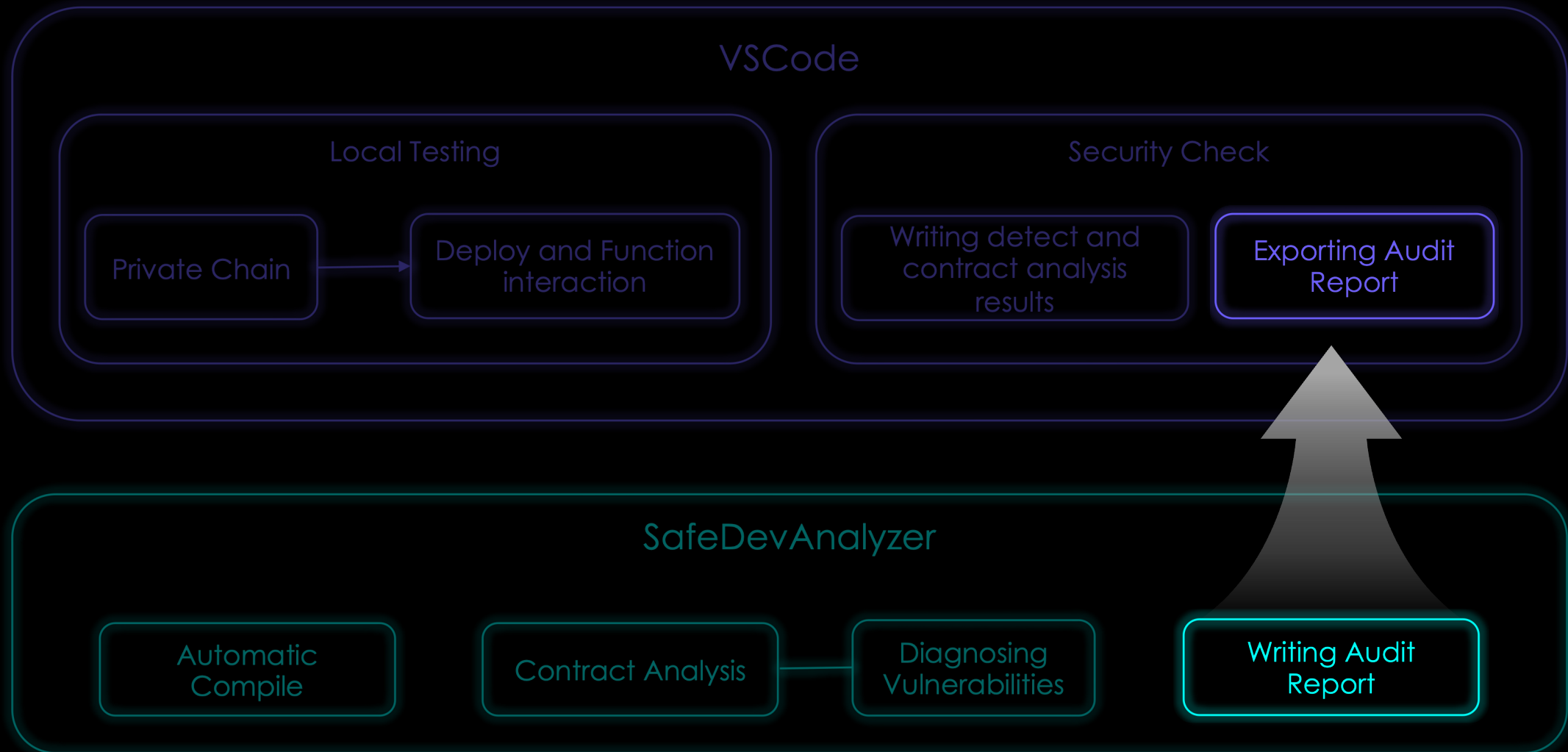
Compile & Interaction



SafeDevAnalyzer



SafeDevAnalyzer



Security Analysis

ANTIBLOCK

> COMPILER AND INTERACTION

> SECURITY ANALYSIS

Select File

etherstore.sol

Select Rule

☒ reentrancy

☐ delegatecall

☒ selfdestruct

☐ incorrect-return

☒ weak-prng

☐ overflow

☒ ownership

☒ tx-origin

Run Analysis

etherstore.sol 1, U

uml | drawio | funcSigs | report | graph (this) | graph | inheritance | parse | flatten

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.7.1;
3
4 contract EtherStore {
5     mapping(address => uint) public balances;
6     mapping(address => uint) public highScores;
7     address public owner;
8
9     modifier onlyOwner {
10         require(tx.origin == owner, "Not owner");
11     }
12
13     ftrace | funcSig
14     function deposit() public payable {
15         balances[msg.sender] += msg.value;
16     }
17
18     ftrace | funcSig
19     function withdraw() public onlyOwner() {
20         uint bal = balances[msg.sender];
21         require(bal > 0);
22
23         (bool sent, ) = msg.sender.call{value: bal}("");
24         require(sent, "Failed to send Ether");
25         balances[msg.sender] = 0;
26     }
27
28     ftrace | funcSig
29     function guess(uint _guess) public {
30         uint answer = uint(
31             keccak256(abi.encodePacked(blockhash(block.number - 1), block.timestamp))
32         );
33         if (_guess == answer) {
34             withdraw();
35         }
36     }
37
38     ftrace | funcSig
39     function getBalance() public view returns (uint) {
40         return address(this).balance;
41     }
42
43     UnitTest stub | dependencies | uml | funcSigs | drawio
44     contract TimeLock {
45         mapping(address => uint) public balances;
46         mapping(address => uint) public lockTime;
```

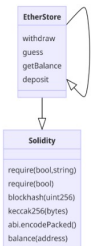
Analysis Result (etherstore.sol)

Dashboard

Korean English

View Details

컨트랙트 분석 요약



취약점 탐지 요약



High 2
중간 1
낮음 0
안내 0

컨트랙트 분석 결과

컨트랙트 이름	상속 관계	상태변수 (State Variable)	함수 정보	외부 호출 함수 (External Call)
Etherstore	N/A	balances(address) highScore(address) owner()	deposit() withdraw() guess(uint256) getBalance()	N/A

취약점 탐지 결과

취약점 탐지기	영향도	신뢰도	코드 라인
reentrancy	High	High	Line 22

설명

- Re-Entrancy 공격은 함수가 이전 실행을 완료하기 전에 컨트랙트 함수를 반복 호출하여 발생합니다. 이로 인해 내부 fallback 함수에서 작성 코드가 실행되어 이더가 탈취될 수 있습니다.

Security Check

Audit Report

2023-11-29

Filename:
/Users/dlanara/Desktop/imm/ProtocolCamp/dev/SafeDevAnalyzer/test/Etherstore.sol

Korean

▼ weak-prng

Detect Results

Detector	Impact	Confidence	Info
weak-prng	High	Medium	EtherStore.guess(uint256) (Etherstore.sol#28-36) uses a weak PRNG: "_guess == answer (Etherstore.sol#33)"

Vulnerability in

line 28: func

line 33:

컨트랙트 분석 결과

컨트랙트 이름	상속 관계	상태변수 (State Variable)	함수 정보	외부 호출 함수 (External Call)
Etherstore	N/A	balances(address) highScore(address) owner()	deposit() withdraw() guess(uint256) getBalance()	N/A
deposit()				
함수 시그니처 (Signature)	가시성 (Visibility)	함수 수정자 (Modifier)	내부 호출 함수 (Internal Call)	외부 호출 함수 (External Call)
0xd0e30db0	public	N/A	N/A	N/A
withdraw()				
함수 시그니처 (Signature)	가시성 (Visibility)	함수 수정자 (Modifier)	내부 호출 함수 (Internal Call)	외부 호출 함수 (External Call)
0x3ccdf60b	public	onlyOwner/throw	return(uint256, uint256) require(bool) onlyOwner	msg.sender.call(value)

Recommendation:

`tx.origin`을 사용하면 다른 컨트랙트에서 `tx.origin`을 사용하는 컨트랙트를 사용할 수 없기 때문에 컨트랙트 간의 상호 운용성이 제한됩니다.
또한, 2016년 7월, Vitalik Buterin이 [ethereum stackexchange](#)에 남긴 답변에 의하면, 향후 이더리움 프로토콜에서 `tx.origin`을 제거할 가능성이 있습니다.

아래와 같이 `tx.origin` 대신 `msg.sender`를 사용하여 인증을 수행하는 것이 안전합니다.

```
function transfer(address payable _to, uint256 _amount) public {
    require(msg.sender == owner);

    (bool sent, ) = _to.call.value(_amount)("");
    require(sent, "Failed to send Ether");
}
```

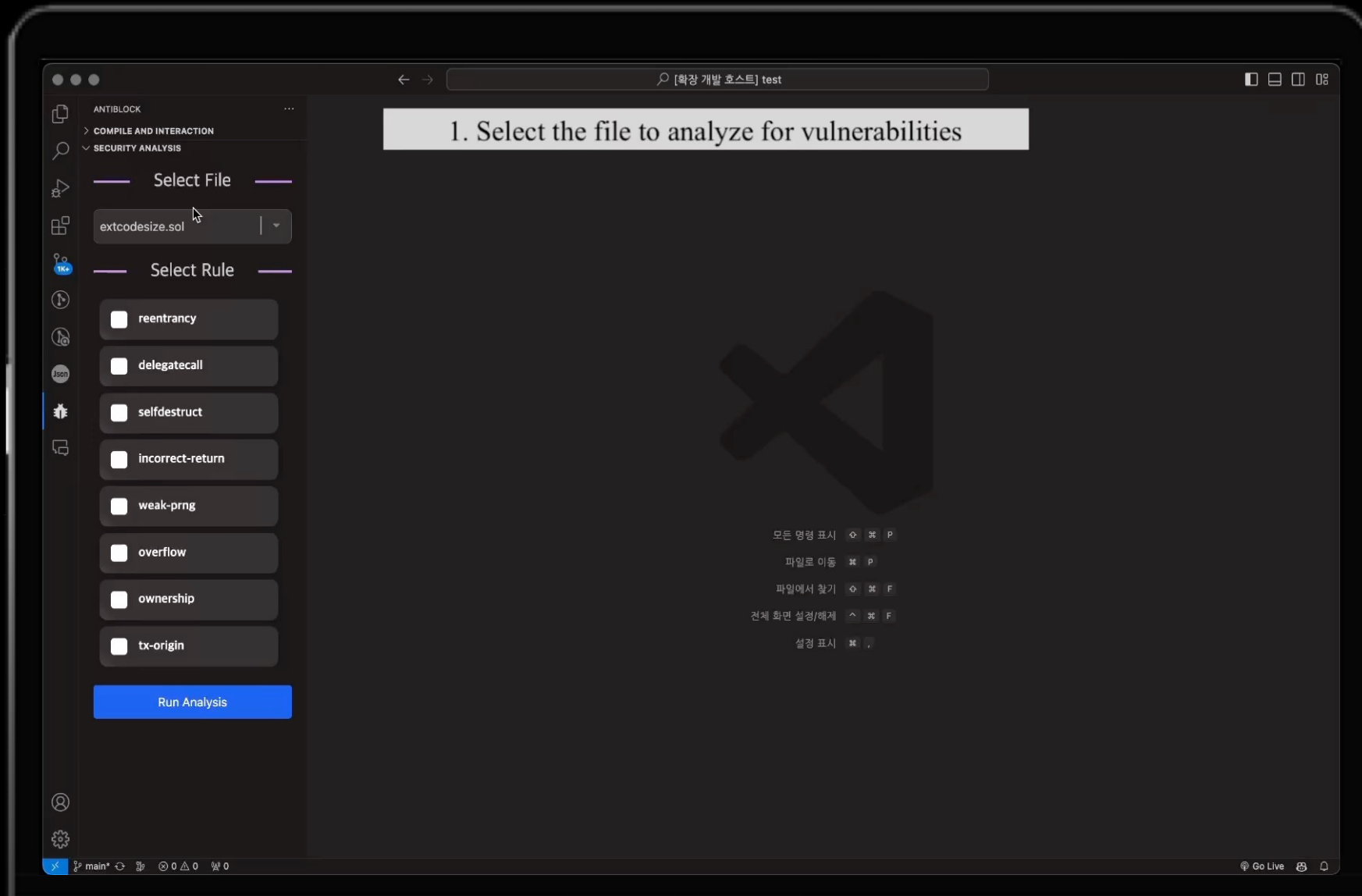
Exploit scenario:

Security Analysis Result

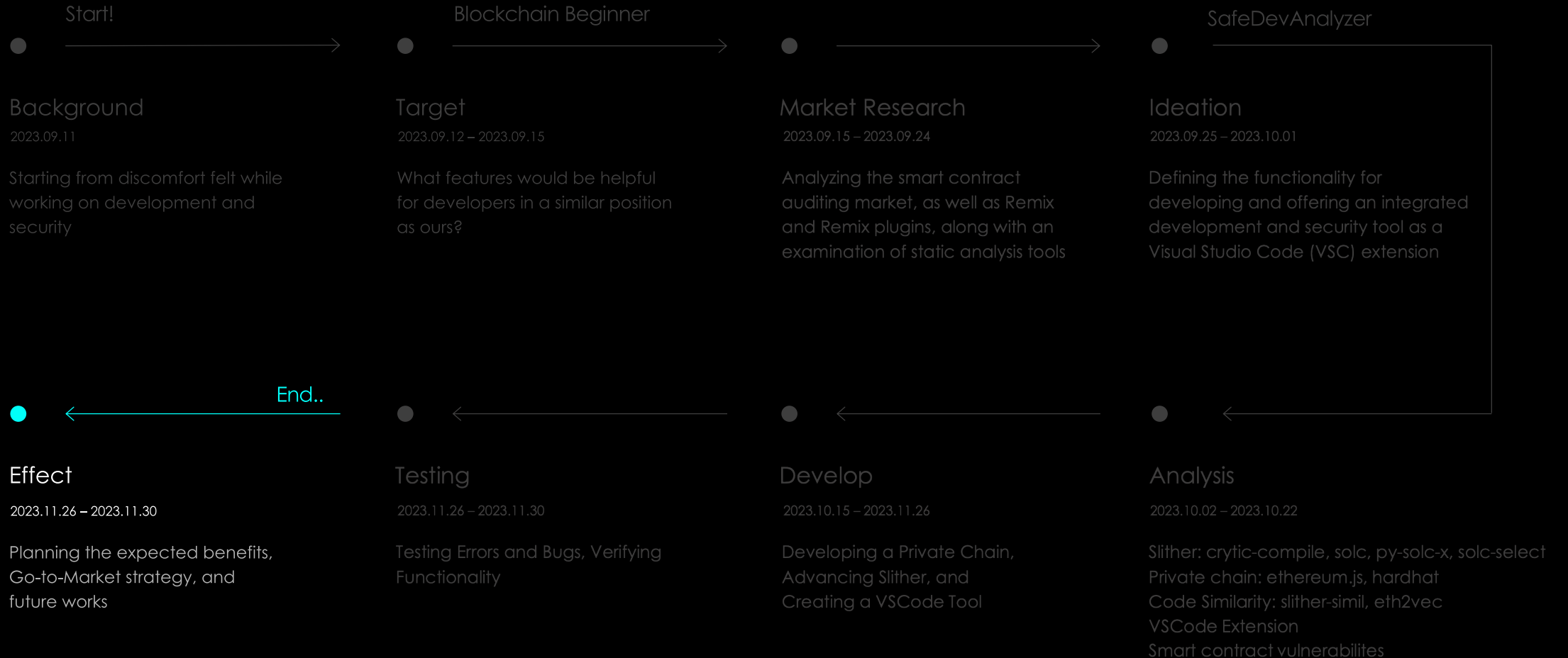
Detector	Impact	Confidence	Code Line
reentrancy	High	High	Line 22
Weak-prng	High	Medium	Line 33
tx-origin	Medium	Medium	Line 10

Alice가 wallet 컨트랙트를 호출하도록 유도한다고 가정합니다.

Security Analysis



Antibug Progress



Effect

Integrated Tools

Integrating development and security tools to allow users to handle all tasks in one place.

Multilingual Support

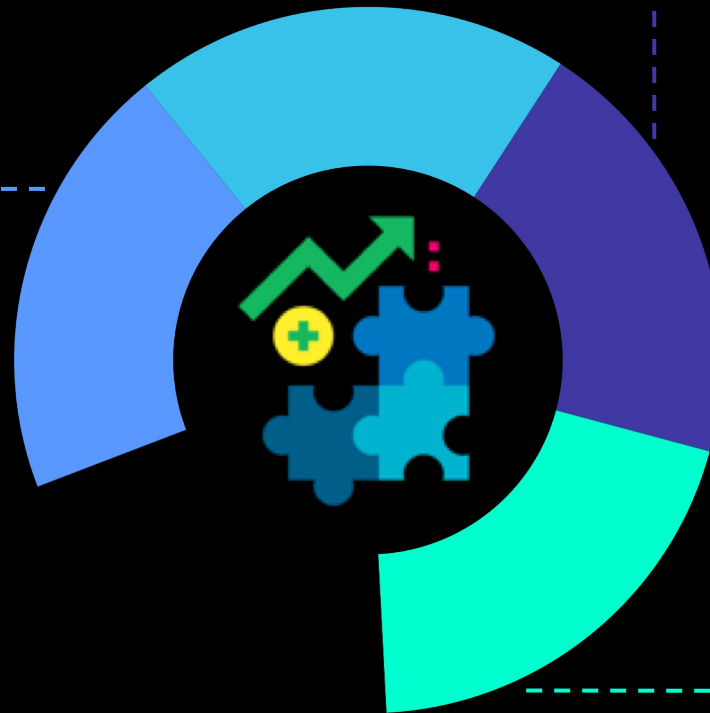
Supporting various languages, including Korean and English, to cater to a diverse user base, considering blockchain's global nature for accessibility by a wide range of users.

Emphasizing Security and Safety

We highlight transparency and immutability in smart contracts and blockchain to enhance reliability and security, rigorously checking security elements.

Simplified Usability

Providing an intuitive UI/UX with detailed product descriptions and usage guidelines to enable easy access and utilization for users.



Strategy and Roadmap

We are striving to improve the technical aspects to provide a better environment and enhanced tools for our users. Our goal is to offer a more advanced environment and improved tools. To achieve this, we are exploring new technical approaches such as vulnerability detection based on code similarity and gas cost optimization using bytecode.



Areas of Improvement

- Gas Cost Estimation & Optimization based on Bytecode
- Tracking data based on storage
- Rollback of a Transaction
- Coverage Improvement Through Rule Enhancement
- Vulnerability Detection based on Code Similarity



Development

Check for Vulnerabilities

Through code similarity

Slither-Simil

Model trained with a dataset of 47,037

✓ Code Element Modification Detection

✓ Code Flow Change Detection

✓ Code Line Omission detection

✓ Comparison Operator Change Detection



SafeDevAnalyzer

Model trained with a dataset of 47,037

✓ Code Element Modification Detection

✓ Code Flow Change Detection

✓ Code Line Omission detection

✓ Comparison Operator Change Detection

Strategy and Roadmap

We aim to introduce and test our tool at events such as grants and hackathons, collecting diverse feedback from users to continuously enhance and improve our project. These activities offer us valuable opportunities to expand the reach of our project.

Our ultimate goal is to create a unified development/security framework. By incorporating user requirements and feedback, we aspire to construct a comprehensive framework that prioritizes essential functionalities and security considerations.



Prototype

- Beta Test & Feedback: Planning product improvements based on user feedback gathered through beta testing.



Development & Security Framework

- Improving Usability and Accessibility
- Tool Integration
- Open Source

Q&A

Thank You :)